

SkillNova

Backend Development – Project Completion Report

Intern Name: **Rohnit**

Organization: **UptoSkills – Intern Management**

Project: **SkillNova – Full-Stack Web Application**

Stack: **FastAPI (Python) · MySQL · SQLAlchemy (Async) · JWT · React**

Report Date: **13 April 2026**

Status: **COMPLETED ✓**

1. Executive Summary

This report documents the successful completion of the **SkillNova backend**, an intern management and course-tracking platform developed as part of the **UptoSkills Internship Programme**. The backend is built with **FastAPI** and connected to a **MySQL** database via **SQLAlchemy's asynchronous session** architecture, ensuring high-performance, non-blocking database operations. The system exposes secure RESTful APIs for user authentication, profile management, and course enrolment, protected by **JWT (JSON Web Token)** Bearer authentication and an **OAuth2 Password** flow.

All API endpoints were tested and verified through the interactive **Swagger UI** documentation auto-generated by FastAPI, with evidence screenshots captured at **localhost:8000**. Database integrity was confirmed through **MySQL Workbench**. The frontend React application (running at **localhost:5173**) successfully communicates with the backend, completing the full authentication loop.

2. Project Overview

Attribute	Details
Project Name	SkillNova
Purpose	Intern skill-tracking & course management platform
Backend Framework	FastAPI (Python 3.11+)
Database	MySQL via SQLAlchemy AsyncSession (aiomysql driver)
Auth Mechanism	JWT Bearer Tokens + OAuth2 Password Flow
Password Hashing	bcrypt (passlib)
API Docs	Swagger UI – http://localhost:8000/docs
Frontend	React 18 – http://localhost:5173
CORS Policy	Configured for localhost:5173 origin
Report Date	13 April 2026

3. Assigned Tasks & Completion Status

#	Task	Details	Status
1	Project Structure	Created modular layout: routes/, models/, schemas/, database.py, main.py	✓ Done
2	MySQL + SQLAlchemy	Async engine (aiomysql), AsyncSession, Base model, init_db() on startup	✓ Done
3	Auth APIs	POST /api/auth/register · POST /api/auth/login · JWT token generation (HS256)	✓ Done
4	Course APIs	GET /api/courses · GET /api/courses/{id} · POST /api/courses/{id}/enroll	✓ Done
5	User Profile APIs	GET /api/profile/ · PUT /api/profile/update — JWT-protected endpoints	✓ Done
6	Auth Middleware	Dependency-injection guard (get_current_user) applied to all protected routes	✓ Done

4. Technical Highlights

Async Architecture: Migrated from synchronous SQLAlchemy to AsyncSession using the aiomysql driver, enabling full non-blocking I/O throughout the database layer.

bcrypt Fix: Resolved a passlib/bcrypt version warning by pinning compatible library versions, ensuring clean startup with no deprecation noise.

CORS Resolution: Configured FastAPI CORSMiddleware to allow the React dev server origin (localhost:5173), eliminating cross-origin request failures.

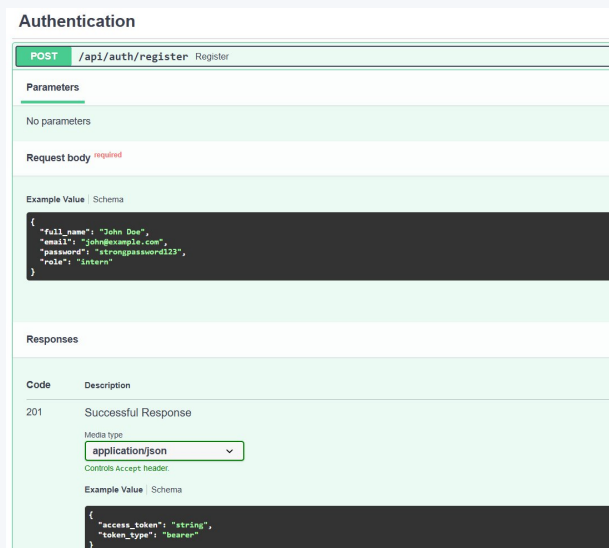
JWT Flow: Implemented HS256 token generation on login/register, with a reusable get_current_user dependency that decodes and validates Bearer tokens on every protected route.

OAuth2 Integration: Swagger UI OAuth2PasswordBearer scheme configured with Token URL at /api/auth/login, enabling one-click in-browser auth testing.

Database Schema: Three tables auto-created via SQLAlchemy metadata: users, courses, enrollments — confirmed visible in MySQL Workbench under the skillnova_db schema.

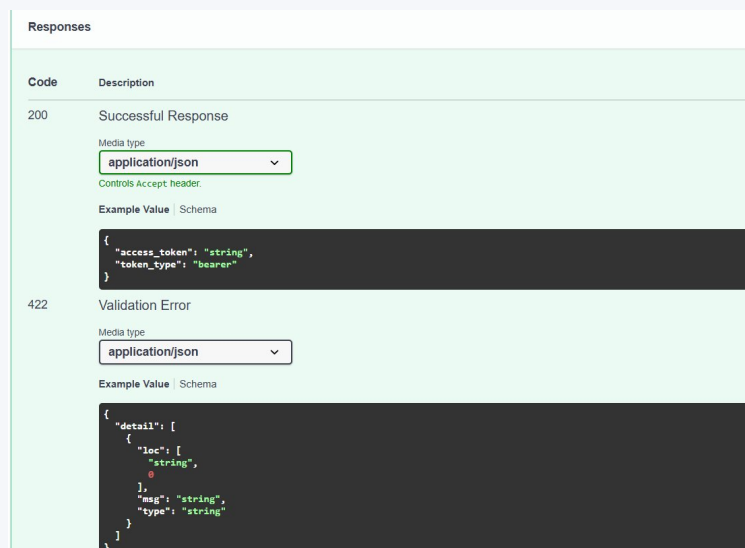
5. Evidence – API & Database Screenshots

Figure 1 – POST /api/auth/register



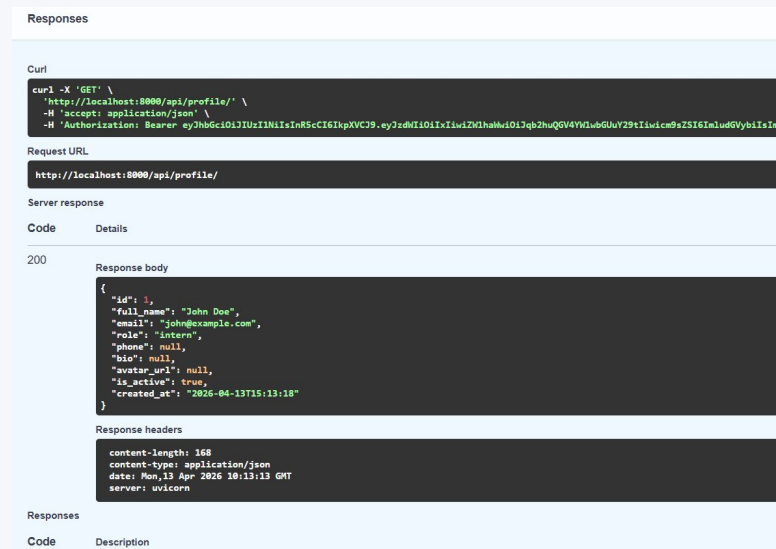
Swagger UI shows the Register endpoint accepting full_name, email, password, and role fields. On success the API returns HTTP 201 with an access_token (Bearer) and token_type.

Figure 2 – POST /api/auth/login (Response Schema)



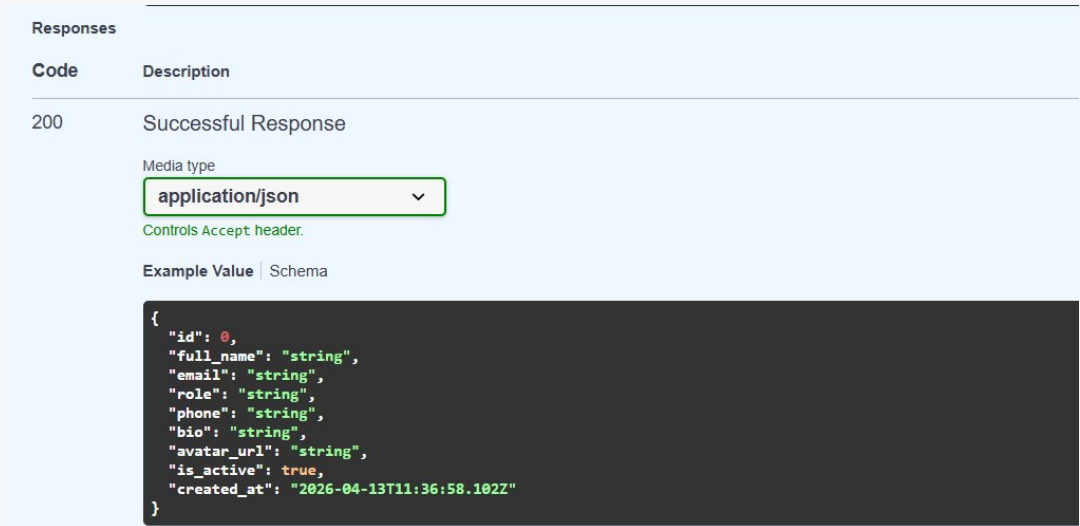
The Login endpoint response schema confirms it returns an access_token string and token_type: 'bearer'. A 422 Validation Error schema is also documented for malformed requests.

Figure 3 – GET /api/profile/ (Live Response)



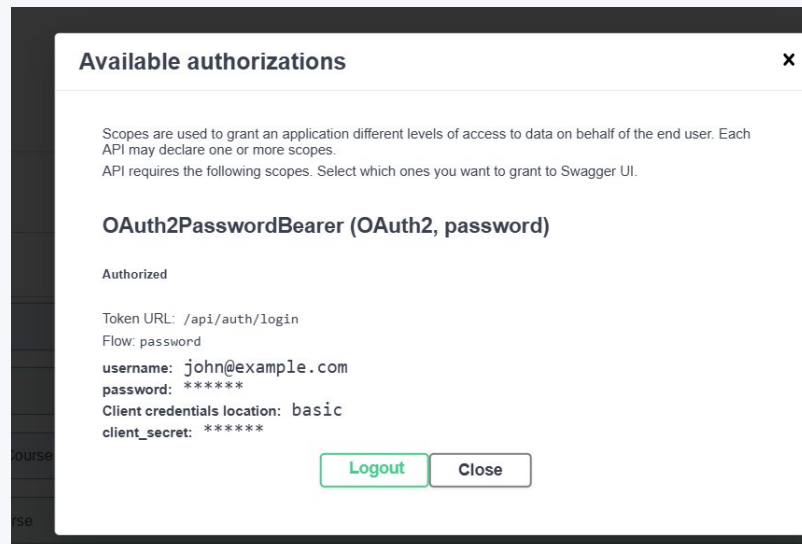
A live authenticated request to GET /api/profile/ via Swagger UI using a Bearer token. The server returns HTTP 200 with user id:1, full_name:'John Doe', email, role:'intern', is_active:true, and created_at timestamp. Server: uvicorn, Date: Mon 13 Apr 2026.

Figure 4 – Profile Endpoint Response Schema



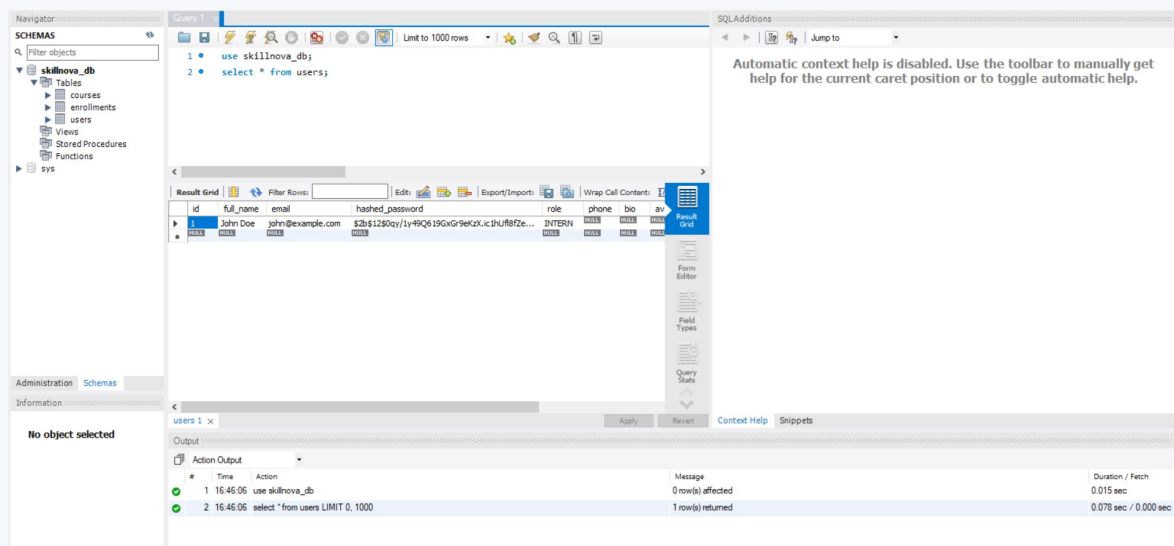
The documented schema for the Profile API response, showing all expected fields: id, full_name, email, role, phone, bio, avatar_url, is_active, and created_at.

Figure 5 – OAuth2 Authorization Confirmed



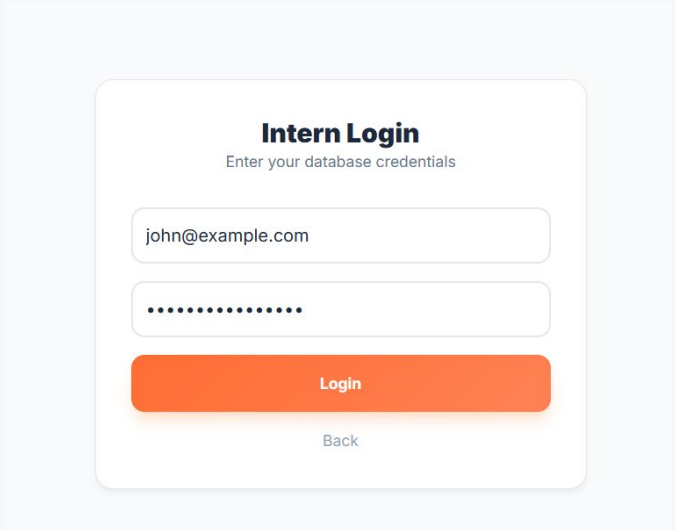
Swagger UI Available Authorizations dialog confirms a successful OAuth2PasswordBearer session. Token URL is /api/auth/login, Flow: password, with user john@example.com authorized and the Authorized label shown in green.

Figure 6 – MySQL Workbench: skillnova_db Users Table



MySQL Workbench running `SELECT * FROM users` on `skillnova_db`. The Navigator shows all three tables: `users`, `courses`, `enrollments`. The result grid shows `id:1`, `full_name:'John Doe'`, `email:'john@example.com'`, `hashed_password (bcrypt hash)`, `role:'INTERN'`. Query executed in 0.078s with 1 row returned.

Figure 7 – React Frontend: Intern Login Portal



The SkillNova React frontend (localhost:5173) displaying the Intern Login screen. The form accepts email and password credentials — demonstrated here with john@example.com / strongpassword123 — which are submitted to the FastAPI backend at POST /api/auth/login to obtain a JWT Bearer token and trigger the dashboard redirect. This confirms successful full-stack integration between the React frontend and FastAPI backend.

6. Conclusion

All six assigned backend tasks have been successfully completed and verified. The **SkillNova FastAPI backend** is fully operational, exposing a secure, well-documented REST API backed by an asynchronous MySQL database layer. Authentication is enforced through JWT Bearer tokens with an OAuth2 password flow, and all protected endpoints correctly reject unauthenticated requests.

The frontend React application at **localhost:5173** successfully connects to the backend at **localhost:8000**, completing the full-stack integration. CORS, bcrypt, and async migration issues encountered during development were diagnosed and resolved, demonstrating strong debugging and architectural capability.

SkillNova Backend · UptoSkills Intern Management · Confidential